

Chapter 2

Basic Parsing Patterns

Language recognition is a critical step in just about any language application. To interpret or translate a phrase, we first have to recognize what kind of phrase it is (sentences are made up of phrases). Once we know that a phrase is an assignment or function call, for example, we can act on it. To recognize a phrase means two things. First, it means we can distinguish it from the other constructs in that language. And, second, it means we can identify the elements and any substructures of the phrase. For example, if we recognize a phrase as an assignment, we can identify the variable on the left of the = and the expression substructure on the right. The act of recognizing a phrase by computer is called *parsing*.

This chapter introduces the most common parser design patterns that you will need to build recognizers by hand. There are multiple parser design patterns because certain languages are harder to parse than others. As usual, there is a trade-off between parser simplicity and parser strength. Extremely complex languages like C++ typically require less efficient but more powerful parsing strategies. We'll talk about the more powerful parsing patterns in the next chapter. For now, we'll focus on the following basic patterns to get up to speed:

- Pattern 1, *Mapping Grammars to Recursive-Descent Recognizers*, on page 45. This pattern tells us how to convert a grammar (formal language specification) to a hand-built parser. It's used by the next three patterns.
- Pattern 2, *LL(1) Recursive-Descent Lexer*, on page 49. This pattern breaks up character streams into tokens for use by the parsers defined in the subsequent patterns.

- Pattern 3, *LL(1) Recursive-Descent Parser*, on page 54. This is the most well-known recursive-descent parsing pattern. It only needs to look at the current input symbol to make parsing decisions. For each rule in a grammar, there is a parsing method in the parser.
- Pattern 4, *LL(k) Recursive-Descent Parser*, on page 59. This pattern augments an *LL(1)* recursive-descent parser so that it can look multiple symbols ahead (up to some fixed number *k*) in order to make decisions.

Before jumping into the parsing patterns, this chapter provides some background material on language recognition. Along the way, we will define some important terms and learn about grammars. You can think of grammars as functional specifications or design documents for parsers. To build a parser, we need a guiding specification that precisely defines the language we want to parse.

Grammars are more than designs, though. They are actually executable “programs” written in a domain-specific language (DSL) specifically designed for expressing language structures. Parser generators such as ANTLR can automatically convert grammars to parsers for us. In fact, ANTLR mimics what we’d build by hand using the design patterns in this chapter and the next.

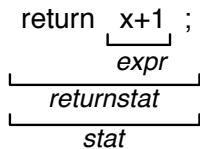
After we get a good handle on building parsers by hand, we’ll rely on grammars throughout the examples in the rest of the book. Grammars are often 10 percent the size of hand-built recognizers and provide more robust solutions. The key to understanding ANTLR’s behavior, though, lies in these parser design patterns. If you have a solid background in computer science or already have a good handle on parsing, you can probably skip this chapter and the next.

Let’s get started by figuring out how to identify the various substructures in a phrase.

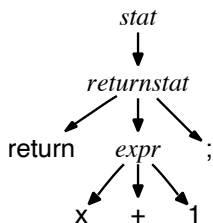
2.1 Identifying Phrase Structure

In elementary school, we all learned (and probably forgot) how to identify the parts of speech in a sentence like *verb* and *noun*. We can do the same thing with computer languages (we call it *syntax analysis*). Vocabulary symbols (*tokens*) play different roles like *variable* and *operator*. We can even identify the role of token subsequences like *expression*.

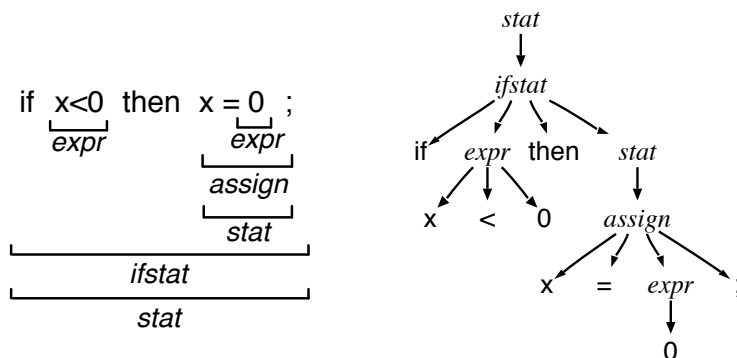
Take `return x+1;`, for example. Sequence `x+1` plays the role of an expression and the entire phrase is a return statement, which is also a kind of statement. If we represent that visually, we get a sentence diagram of sorts:



Flip that over, and you get what we call a *parse tree*:



Tokens hang from the parse tree as leaf nodes, while the interior nodes identify the phrase substructures. The actual names of the substructures aren't important as long as we know what they mean. For a more complicated example, take a look at the substructures and parse tree for an if statement:



Parse trees are important because they tell us everything we need to know about the syntax (*structure*) of a phrase. To parse, then, is to conjure up a two-dimensional parse tree from a flat token sequence.

2.2 Building Recursive-Descent Parsers

A parser checks whether a sentence conforms to the syntax of a *language*. (A language is just a set of valid sentences.) To verify language membership, a parser has to identify a sentence's parse tree. The cool thing is that the parser doesn't actually have to construct a tree data structure in memory. It's enough to just recognize the various substructures and the associated tokens. Most of the time, we only need to execute some code on the tokens in a substructure. In practice, we want parsers to “do this when they see that.”

To avoid building parse trees, we trace them out implicitly via a function call sequence (a call tree). All we have to do is make a function for each named substructure (interior node) of the parse tree. Each function, say, *f*, executes code to match its children. To match a substructure (subtree), *f* calls the function associated with that subtree. To match token children, *f* can call a `match()` support function. Following this simple formula, we arrive at the following functions from the parse tree for `return x+1;`:

```
/** To parse a statement, call stat(); */
void stat()      { returnstat(); }
void returnstat() { match("return"); expr(); match(";"); }
void expr()      { match("x"); match("+"); match("1"); }
```

Function `match()` advances an input cursor after comparing the current input token to its argument. For example, before calling `match("return")`, the input token sequence looks like this:

```
return x + 1 ;
  ↑
```

`match("return")` makes sure that current (first) token is `return` and advances to the next (second) token. When we advance the cursor, we *consume* that token since the parser never has to look at it again. We can represent consumed tokens with a dark gray box:

```
return x + 1 ;
  ↑
```

To make things more interesting, let's figure out how to parse the three kinds of statements found in our parse trees: **if**, **return**, and assignment statements. To distinguish what kind of statement is coming down the road, `stat()` needs to branch according to the token under the input

cursor (the light gray box). We call that token the *lookahead token*. Here's one way to implement `stat()`:

```
void stat() {
    if ( «lookahead token is return» )      returnstat();
    else if ( «lookahead token is identifier» ) assign();
    else if ( «lookahead token is if» )      ifstat();
    else «parse error»
}
```

We call this kind of parser a *top-down parser* because it starts at the top of the parse tree and works its way down to the token leaf nodes.

More specifically, we're using Pattern 3, *LL(1) Recursive-Descent Parser*, on page 54. *Descent* refers to its top-down nature, and *recursive* refers to the fact that its functions potentially call themselves. For example, in Section 2.1, *Identifying Phrase Structure*, on page 38, the *stat* substructure for the assignment statement appears within (under) the *stat* for the if statement. Nesting in a parse tree begets recursion in a recursive-descent parser. The formal designation for a top-down parser that uses a single lookahead token is *LL(1)*. The first *L* means "read the input from left to right." The second *L* means "descend into parse tree children from left to right." For complicated languages, we can use more lookahead yielding Pattern 4, *LL(k) Recursive-Descent Parser*, on page 59.

At this point, we've got a parser that can recognize the various substructures using different functions. But, that's all it does. To execute application-specific code upon seeing a particular substructure, we have to add code to the appropriate function. For example, if we want to print "found return" every time we see a return statement, we can add the following anywhere in `returnstat()`:

```
System.out.println("found return");
```

The patterns that follow this introductory material fill in some details, but that's really all there is to building parsers: predicting which kind of phrase approaches, invoking functions to match substructures, matching tokens, and executing application-specific actions (code).

The first few recursive-descent parsers you build are pretty fun. After a while, though, building them is monotonous. The parsing functions are so similar and consistent that we can easily generate them automatically. The only problem is describing the structure of our language to the computer. Since most languages have an infinite number of sentences, we can't just delineate them. For the same reason, we can't

delineate all possible parse trees. What we need is a DSL for specifying languages.

2.3 Parser Construction Using a Grammar DSL

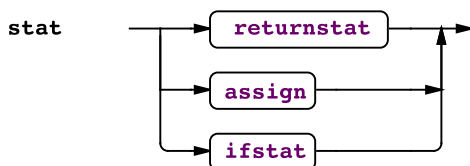
Building recursive-descent parsers in a general-purpose programming language is tedious and error-prone. We have to type the same code templates over and over again. It's much more productive to use a DSL specifically designed for describing languages. “Programs” in this DSL are called *grammars*. Tools that translate grammars to parsers are called *parser generators*. Grammars are concise and act like functional specifications for languages. They are much easier to read than the equivalent recursive-descent parser implementations.

Substructures in the parse tree and functions in the parser correspond to *rules* in a grammar. The children of a substructure become references to rules and tokens on the right side of a rule definition. The if-then-else code template in the parser becomes a | separated list of alternative substructures. Here is one way to encode the recursive-descent parser from the previous section as an ANTLR grammar:

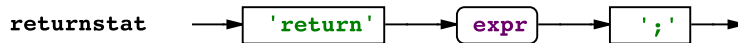
```
stat      : returnstat      // "return x+0;" or
          | assign          // "x=0;" or
          | ifstat          // "if x<0 then x=0;"
          ;
returnstat : 'return' expr ';' ; // single-quoted strings are tokens
assign     : 'x' '=' expr ;
ifstat     : 'if' expr 'then' stat ;
expr       : 'x' '+' '0'      // used by returnstat
          | 'x' '<' '0'       // used by if conditional
          | '0'              // used in assign
          ;
```

This example foreshadows much of Pattern 1, *Mapping Grammars to Recursive-Descent Recognizers*, on page 45.

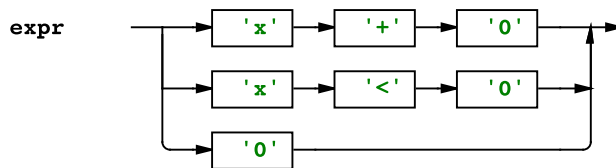
Read rule **stat** as “A **stat** can be either a **returnstat**, an **assign**, or an **ifstat**.” We can use a *syntax diagram* to visualize the control flow within that rule (using ANTLRWorks):



If there is only one alternative, such as in rule **returnstat**, the syntax diagram shows just a sequence of elements:



When rules get more complicated, syntax diagrams come in handy. For example, here is the syntax diagram for rule **expr**:



Rule **expr** might look a little funny to you, and it should. It says that expressions can only be one of three alternative token sequences. Further, variables can only be `x`, and integers can only be `0`. This brings us to the last piece of the language recognition puzzle: combining input characters into vocabulary symbols (tokens).

2.4 Tokenizing Sentences

Humans unconsciously combine letters into words before recognizing grammatical structure while reading. As adults, we are so good at it that we don't notice it. Beginning readers, on the other hand, move along with their fingers trying to sound out the words.

Reading Morse code exposes this hidden process nicely by forcing us to move character by character. Before interpreting a sentence as English, we have to convert the dots and dashes to letters and then combine the letters into words. Once we have words, we can apply English grammatical structure. For example, here is `print 34` in Morse code:

```

.-.-. .-.-. .-.-. -.-. -.-.-.-. ....-
p    r    i    n    t 3    4

```

Recognizers that feed off character streams are called *tokenizers* or *lexers* (see Pattern 2, *LL(1) Recursive-Descent Lexer*, on page 49). Just as an overall sentence has structure, the individual tokens have structure. At the character level, we refer to syntax as the *lexical structure*.

Grammars describe language structures, and so we can also use them for lexical specifications.

For example, here are suitable definitions for numbers and identifiers:

```
Number : '0'..'9'+ ;           // 1-or-more digits (0..9)
ID      : ('a'..'z'|'A'..'Z')+ ; // 1-or-more upper or lower case letters
```

Now we can make a better version of rule **expr** using generic variable names and numbers instead of specific ones:

```
expr      : ID '+' Number // used by returnstat
          | ID '<' Number  // used by if conditional
          | Number        // used in assign
          ;
```

This rule still isn't general enough, but it demonstrates how lexical and syntactic rules can interact.

To make things more concrete, let's look at a real grammar for a simple language. Say we want to recognize lists of names such as [a,b,c] and nested lists such as [a,[b,c],d]. We could use the following ANTLR grammar with three syntactic rules and one lexical rule (lexical rules start with an uppercase letter):

[Download](#) `parsing/topdown/NestedNameList.g`

```
grammar NestedNameList;
list      : '[' elements ']';           // match bracketed list
elements : element (',' element)* ;    // match comma-separated list
element   : NAME | list ;              // element is name or nested list
NAME      : ('a'..'z'|'A'..'Z')+ ;    // NAME is sequence of >=1 letter
```

We can see the parse trees for those two sentences in Figure 2.1, on the next page. The leaves of the parse tree (highlighted nodes) are the tokens from the input stream. The interior nodes are rule names from the grammar. Pattern 2, *LL(1) Recursive-Descent Lexer*, on page 49 and Pattern 3, *LL(1) Recursive-Descent Parser*, on page 54 show how to build a lexer and parser for this grammar.

When reading the lexer and parser design patterns, you'll notice that they are nearly identical. This makes sense given that both patterns look for structure in input sequences. The only difference lies in the type of their input symbols, characters or tokens. We can even take this one step further. Pattern 14, *Tree Grammar*, on page 134 recognizes structure in tree node sequences.

With this preparatory grammar and parsing discussion out of the way, we can define four classic parsing patterns. After looking at them, we'll be able to build parsers for lots of languages. Ultimately we'll use ANTLR grammars rather than hand-built recursive-descent parsers because it's a lot easier. Knowledge of these parsing design patterns still

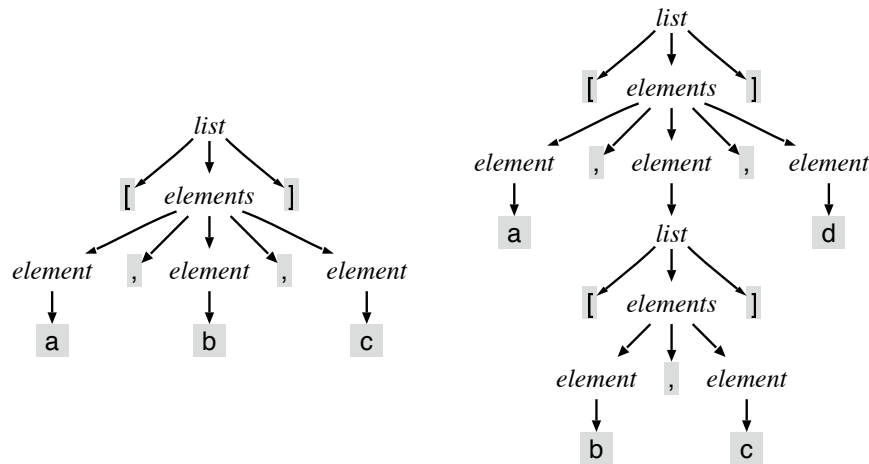


Figure 2.1: Parse trees for [a,b,c] and [a,[b,c],d] with highlighted token leaves

matters, though, because grammars are what drive our parser generators. We still need a good understanding of the underlying mechanism to build grammars effectively.

1

Mapping Grammars to Recursive-Descent Recognizers



Purpose

This translates a grammar to a recursive-descent recognizer that matches phrases and sentences in the language specified by the grammar.

This pattern identifies the core control-flow framework for any recursive-descent lexer, parser, or tree parser.

Discussion

Even when building lexers and parsers by hand, the best starting point is a grammar. Grammars are a very concise way to express the languages you intend to recognize. Not only that, but grammars are excellent documentation that can go into a reference manual and into

parsing code as comments. This pattern gives us a way to build recognizers directly from grammars.

A word of caution: this pattern works for many but not all grammars. The most obvious troublesome grammar construct is *left recursion* (a rule that invokes itself without consuming a token). Left recursion results in an infinite method invocation loop. For example, the following rule yields a parser that does not terminate:

```
r : r X ;
```

Using this design pattern, we'd end up with a function that immediately called itself, leading to an infinite loop:

```
void r() { r(); match(X); }
```

Besides left-recursive rules, there are other grammar constructs that yield *nondeterministic* recursive-descent recognizers. A nondeterministic recognizer cannot decide which path to take. The parsers in Pattern 4, *LL(k) Recursive-Descent Parser*, on page 59 and Pattern 5, *Backtracking Parser*, on page 71, use more and more lookahead to increase the strength of the recognizer. The more powerful the underlying recognition strategy, the easier it is to write a grammar. That is because more powerful parsing strategies allow a larger set of grammars.

Implementation

A grammar, $\mathcal{G}$, is a set of rules from which we generate a class definition (in any object-oriented programming language) containing a method for each rule:

```
public class G extends Parser { // parser definition written in Java
    «token-type-definitions»
    «suitable-constructor»
    «rule-methods»
}
```

Class `Parser` defines the state a typical parser needs, such as a lookahead token or tokens and an input stream.

Converting Rules

For each rule, r , defined in a grammar, we build a method of the same name:

```
public void r() {
    ...
}
```

The inside of a rule looks exactly like a subrule (a rule embedded within another rule), as we'll see shortly.

Rule references to *r* become method calls: *r()*.

Converting Tokens

Token references for token type *T* become calls to *match(T)*. *match()* is a support method in *Parser* that consumes a token if *T* is the current lookahead token. If there is a mismatch, *match()* throws an exception.

Also, we need to define token type *T* somewhere, either in the parser object or in our lexer object. For every token *T*, we write this:

```
public static final int T = «sequential-integer»;
```

We'll also probably need this:

```
public static final int INVALID_TOKEN_TYPE = 0; // to be explicit
public static final int EOF = -1;               // EOF token type
```

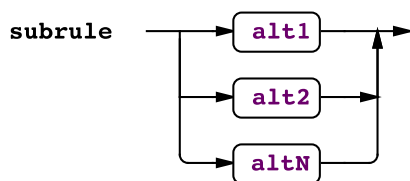
You might wonder why I'm not using the **enum** functionality in Java to represent sets of integers. It's because I always end up wanting to treat them as simple integers.

Converting Subrules

Alternatives become either a switch or an if-then-else sequence, depending on the complexity of the lookahead decision. Each alternative gets an expression that predicts whether that alternative would succeed at the current input location. Consider the following generic subrule:

```
(«alt1»|«alt2»|..|«altN»)
```

The control flow looks like this:



The most general subrule implementation looks like this:

```
if ( «lookahead-predicts-alt1» ) { «match-alt1» }
else if ( «lookahead-predicts-alt2» ) { «match-alt2» }
...
else if ( «lookahead-predicts-altN» ) { «match-altN» }
else «throw-exception» // parse error (no viable alternative)
```

If all of those lookahead expressions only test a single symbol lookahead, we can generate a switch statement, which is usually more efficient:

```
switch ( «lookahead-token» ) {
    case «token1-predicting-alt1» :
    case «token2-predicting-alt1» :
    ...
        «match-alt1»
        break;
    case «token1-predicting-alt2» :
    case «token2-predicting-alt2» :
    ...
        «match-alt2»
        break;
    ...
    case «token1-predicting-altN» :
    case «token2-predicting-altN» :
    ...
        «match-altN»
        break;
    default : «throw-exception»
}
```

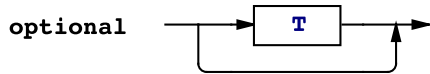
As an optimization, we can collapse subrules whose alternatives are token references such as $(A|B|C)$ into sets. Testing the current symbol of lookahead against a set is usually much faster and smaller than a switch.

All recursive-descent recognizers make decisions according to this template. To implement a recognizer, we fill in those lookahead prediction expressions. The nature of the expressions dictates the strength of the strategy. Pattern 2, *LL(1) Recursive-Descent Lexer*, on the next page and Pattern 3, *LL(1) Recursive-Descent Parser*, on page 54 have *LL(1)* decisions, which means their prediction expressions test one symbol of lookahead. Pattern 4, *LL(k) Recursive-Descent Parser*, on page 59 has *LL(k)* decisions whose prediction expressions test k symbols of lookahead. Pattern 5, *Backtracking Parser*, on page 71 and Pattern 7, *Predicated Parser*, on page 84 augment *LL(k)* decisions with an arbitrary amount of lookahead and arbitrary user-defined run-time tests, respectively.

Converting Subrule Operators

Optional subrules are easy to convert because all we have to do is remove the default error clause from the subrules in the previous sec-

tion. If the subrule is optional, there is no possibility of an error. For example, the control flow of optional subrule (T)? looks like this:



In code, these subrules become conditional statements. For example, (T)? becomes the following:

```
if ( «lookahead-is-T» ) { match(T); } // no error else clause
```

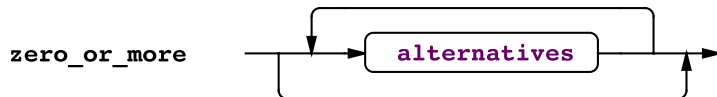
The control flow of one or more (...) + subrules looks like this:



In code, (...) + subrules become do-while loops:

```
do {
    «code-matching-alternatives»
} while ( «lookahead-predicts-an-alt-of-subrule» );
```

Zero or more (...) * subrules are like optional one-or-more loops. Their control flow looks like this:



In code, (...) * subrules become while loops:

```
while ( «lookahead-predicts-an-alt-of-subrule» ) {
    «code-matching-alternatives»
}
```

This way, the recognizer can skip over the subrule if the lookahead does not predict an alternative within the subrule.

In the next pattern, we will look at how to build lexers using this grammar-to-recognizer mapping as scaffolding.

2

LL(1) Recursive-Descent Lexer

**Purpose**

Lexers derive a stream of tokens from a character stream by recognizing lexical patterns.

Lexers are also called *scanners*, *lexical analyzers*, and *tokenizers*. As a bonus, this pattern can recognize nested lexical structures such as nested comments (even though languages typically don't have super-complicated lexical structure).

Discussion

The goal of the lexer is to emit a sequence of tokens. Each token has two primary attributes: a *token type* (symbol category) and the text associated with it. In the English language, we've got categories such as verbs and nouns as well as punctuation symbols such as commas and periods. All words within a particular category are said to have the same token type, though their associated text is different.

Let's identify the token types from the list language in Section 2.4, *Tokenizing Sentences*, on page 43. The token type **NAME** represents the identifier category. Then we need token types for the fixed string vocabulary symbols: **COMMA**, **LBRACK**, and **RBRACK**. Lexers also typically deal with whitespace and comments. Because the parser ignores these, we don't bother defining token types for them. We can just have the lexer throw out such character sequences if it finds them between tokens.

To build a lexer by hand, we write a method for each token definition (lexical rule). In other words, token **T**'s definition becomes method **T**(**l**). These methods recognize the pattern expressed in the associated lexical rule. In programming languages, for example, there are methods to match integers, floating-point numbers, identifiers, operators, and so on. To write the code for the lexer rules, we follow Pattern 1, *Mapping Grammars to Recursive-Descent Recognizers*, on page 45.

To make the lexer look like an enumeration of tokens, it's handy to define a method called `nextToken()`. `nextToken()` uses the lookahead character (character under the input cursor) to route control flow to the appropriate recognition method. For example, upon seeing a letter, `nextToken()` would call a method to recognize the identifier pattern. Here

is the core of a typical lexer `nextToken()` that skips whitespace and comments:

```
public Token nextToken() {
    while ( «lookahead-char» != EOF ) { // EOF == -1 per java.io
        if ( «comment-start-sequence» ) { COMMENT(); continue; }
        ... // other skip tokens
        switch ( «lookahead-char» ) { // which token approaches?
            case «whitespace» : { consume(); continue; } // skip
            case «chars-predicting-T1» : return T1(); // match T1
            case «chars-predicting-T2» : return T2();
            ...
            case «chars-predicting-Tn» : return Tn();
            default : «error»
        }
    }
    return «EOF-token»; // return token with EOF_TYPE token type
}
```

To use this lexer pattern, we create an instance of a lexer from an input string or stream reader. Our parser object then feeds off this lexer, calling its `nextToken()` method to extract tokens. The code would look something like this:

```
MyLexer lexer = new MyLexer("«input-sentence»"); // create lexer
MyParser parser = new MyParser(lexer); // create parser
parser.«start_rule»(); // begin parsing, looking for a list sentence
```

Implementation

As a sample implementation, let's build a lexer for the nested list-of-names grammar shown in Section 2.4, *Tokenizing Sentences*, on page 43. Our goal is a lexer that we can treat like an enumeration. Here is a loop from the test rig that pulls tokens out of the lexer until it returns a token with type `EOF_TYPE`:

[Download parsing/lexer/Test.java](#)

```
ListLexer lexer = new ListLexer(args[0]);
Token t = lexer.nextToken();
while ( t.type != Lexer.EOF_TYPE ) {
    System.out.println(t);
    t = lexer.nextToken();
}
System.out.println(t); // EOF
```

We'd like output like this from the list argument:

```
$ java Test '[a, b ]'
<'[,LBRACK>
<'a',NAME>
<',' ,COMMA>
```

```

<'b',NAME>
<']',RBRACK>
<'<EOF>',<EOF>>
$

```

To implement this, we'll need Token objects, an abstract Lexer for support code, and finally a concrete ListLexer to do the actual work. Let's start by defining tokens to have token type and text properties.

[Download](#) parsing/lexer/Token.java

```

public class Token {
    public int type;
    public String text;
    public Token(int type, String text) {this.type=type; this.text=text;}
    public String toString() {
        String tname = ListLexer.tokenNames[type];
        return "<'"+text+"',"+tname+">";
    }
}

```

For now, let's skip over the support code, Lexer, to focus on the list language lexer. We need to define token types, and we might as well define them in our lexer:

[Download](#) parsing/lexer/ListLexer.java

```

public class ListLexer extends Lexer {
    public static int NAME = 2;
    public static int COMMA = 3;
    public static int LBRACK = 4;
    public static int RBRACK = 5;
    public static String[] tokenNames =
        { "n/a", "<EOF>", "NAME", "COMMA", "LBRACK", "RBRACK" };
    public String getTokenName(int x) { return tokenNames[x]; }

    public ListLexer(String input) { super(input); }
    boolean isLETTER() { return c>='a'&&c<='z' || c>='A'&&c<='Z'; }
}

```

The getTokenName() method helps us generate good error messages and generate readable Token.toString() output.

Following the nextToken() code template, here is the method matching tokens or routes traffic to the appropriate method:

[Download](#) parsing/lexer/ListLexer.java

```

public Token nextToken() {
    while ( c!=EOF ) {
        switch ( c ) {
            case ' ': case '\t': case '\n': case '\r': WS(); continue;
            case ',': consume(); return new Token(COMMA, ",");
            case '[': consume(); return new Token(LBRACK, "[" );

```



```

        case ']' : consume(); return new Token(RBRACK, "];");
    default:
        if ( isLETTER() ) return NAME();
        throw new Error("invalid character: "+c);
    }
}
return new Token(EOF_TYPE, "<EOF>");
}

```

To match an identifier, we need to buffer characters while the lookahead character is a letter:

[Download](#) parsing/lexer/ListLexer.java

```

/** NAME : ('a'..'z' | 'A'..'Z')+; // NAME is sequence of >=1 letter */
Token NAME() {
    StringBuilder buf = new StringBuilder();
    do { buf.append(c); consume(); } while ( isLETTER() );
    return new Token(NAME, buf.toString());
}

```

If nextToken() finds a whitespace character, it calls WS() to consume all the whitespace without buffering it up or returning a token. Here is WS():

[Download](#) parsing/lexer/ListLexer.java

```

/** WS : (' ' | '\t' | '\n' | '\r')* ; // ignore any whitespace */
void WS() {
    while ( c==' ' || c=='\t' || c=='\n' || c=='\r' ) consume();
}

```

On to the support code in our Lexer base class. Here are the necessary fields to maintain state:

[Download](#) parsing/lexer/Lexer.java

```

public abstract class Lexer {
    public static final char EOF = (char)-1; // represent end of file char
    public static final int EOF_TYPE = 1;   // represent EOF token type
    String input; // input string
    int p = 0;    // index into input of current character
    char c;       // current character
}

```

The constructor records the input string and primes the lookahead by loading the first character into lookahead character c.

[Download](#) parsing/lexer/Lexer.java

```

public Lexer(String input) {
    this.input = input;
    c = input.charAt(p); // prime lookahead
}

```

Method `consume()` advances the input pointer and sets lookahead character `c` to the next character in the string. This method also detects when the lexer has run out of input characters, setting the lookahead character to EOF (-1).

[Download](#) parsing/lexer/Lexer.java

```
/** Move one character; detect "end of file" */
public void consume() {
    p++;
    if ( p >= input.length() ) c = EOF;
    else c = input.charAt(p);
}

/** Ensure x is next character on the input stream */
public void match(char x) {
    if ( c == x ) consume();
    else throw new Error("expecting "+x+"; found "+c);
}
```

Class `Lexer` is abstract because it has no code to match tokens. Any concrete subclasses, such as our `ListLexer`, need to implement the following methods:

[Download](#) parsing/lexer/Lexer.java

```
public abstract Token nextToken();
public abstract String getTokenName(int tokenType);
```

Related Patterns

There is a great similarity between the structure of this lexer and the structure of Pattern 3, *LL(1) Recursive-Descent Parser*. They're both instances of recursive-descent recognizers generated using Pattern 1, *Mapping Grammars to Recursive-Descent Recognizers*, on page 45.

3

LL(1) Recursive-Descent Parser



Purpose

This pattern analyzes the syntactic structure of the token sequence of a phrase using a single lookahead token.

This parser belongs to the *LL(1)* top-down parser class in particular because it uses a single token of lookahead (hence the “1” in the name). It's the core mechanism of all subsequent parsing patterns.

Discussion

This pattern shows how to implement parsing decisions that use a single token of lookahead. It's the weakest form of recursive-descent parser but the easiest to understand and implement. If you can conveniently implement your language with this *LL(1)* pattern, you should do so. Pattern 4, *LL(k) Recursive-Descent Parser*, on page 59 uses multisymbol lookahead, which is more powerful but has more complicated infrastructure.

To implement an *LL(1)* recursive-descent parser, we can start by filling in the lookahead expressions from the parsing decisions shown in Pattern 1, *Mapping Grammars to Recursive-Descent Recognizers*, on page 45. To make parsing decisions, the parser tests the current lookahead token against the alternatives' lookahead sets. A lookahead set is the set of tokens that can begin a particular alternative. The parser should attempt the alternative that can start with the current lookahead token. In the next two sections, we'll figure out how to compute lookahead sets and how to detect decisions with more than one viable path.

Computing Lookahead Sets

Formally, we compute lookahead sets using two computations: *FIRST* and *FOLLOW*. In practice, though, it's easier to simply ask ourselves, "What tokens can possibly start phrases beginning at this alternative?" The exact definition of *FIRST* is a bit much to chew on for our purposes here, but if you're interested, a web search reveals plenty of decent descriptions.¹

Let's start with the easiest lookahead computation case: an alternative that begins with a token reference. Its lookahead set is just that token. For example, here is a programming language statement rule where each alternative begins with single token reference:

```
stat: 'if' ...    // lookahead set is {if}
    | 'while' ... // lookahead set is {while}
    | 'for' ...   // lookahead set is {for}
    ;
```

1. <http://www.cs.virginia.edu/~cs415/reading/FirstFollowLL.pdf>

If an alternative begins with a rule reference instead of a token reference, the lookahead set is whatever begins any alternative of that rule.

Here is a rule whose first alternative invokes **stat**:

```
body_element
: stat      // lookahead is {if, while, for}
| LABEL ':' // lookahead is {LABEL}
;
```

The lookahead set for the first alternative is the union of the lookahead sets from **stat**. Lookahead computations only get complicated when we consider empty alternatives. For example, it's not immediately obvious what tokens predict the empty alternative in the following rule:

```
optional_init
: '=' expr
| // empty alternative
;
```

The lookahead for the first alternative is `=`. For the empty alternative, the lookahead is the set of tokens following references to **optional_init**. So, let's add some other rules that invoke **optional_init**:

```
decl: 'int' ID optional_init ';' ;
arg : 'int' ID optional_init ;
func_call: ID '(' arg ')' ; // calls arg; ')' included in lookahead
```

In this case, `;` follows **optional_init** in **decl** so we know that at least `;` is in the set. Rule **arg** also references **optional_init**, but there is no token following it. This means that we have to include whatever follows **arg**. Token `)` can follow a reference to **arg**, so tokens `'` and `)` can follow a reference to **optional_init**.

Don't worry if you don't fully understand these lookahead computations in detail. Most likely you won't have to compute them yourself; ANTLR likes nothing better than to compute these for you. With this general understanding of lookahead computation in mind, let's consider what happens when the same token predicts more than one alternative.

Deterministic Parsing Decisions

LL parsing decisions work only when the lookahead sets predicting the alternatives are disjoint (the sets have no tokens in common).

For example, here is a rule whose parsing decision is deterministic because the single lookahead token uniquely predicts which alternative to choose:

```
/** Match -3, 4, -2.1 or x, salary, username, and so on */
expr: '-'? (INT|FLOAT) // '-', INT, or FLOAT predicts this alternative
    | ID               // ID predicts this alternative
    ;
```

Upon `-`, `INT`, or `FLOAT`, rule `expr` knows to predict the first alternative. Upon `ID`, it knows to predict the second.

If the lookahead sets overlap, though, the parser is *nondeterministic*—it cannot determine which alternative to choose. For example, here's a rule that's nondeterministic for an *LL(1)* parser:

```
expr: ID '++' // match "x++"
    | ID '--' // match "x--"
    ;
```

The two alternatives begin with the same token (`ID`). The token beyond dictates which alternative phrase is approaching. In other words, `expr` is *LL(2)*. An *LL(1)* parser can't see past the left common prefix with only one symbol of lookahead. Without seeing the suffix operator after the `ID`, the parser cannot predict which alternative will succeed. To handle grammatical constructs like this, either tweak your grammar or use Pattern 4, *LL(k) Recursive-Descent Parser*, on page 59. By left-factoring out the common `ID` left-prefix, we get an *LL(1)* grammar that matches the same language:

```
expr: ID ('++' | '--') ; // match "x++" or "x--"
```

If you plan on building lots of parsers by hand, it's worth spending time to get good at computing lookahead sets. Otherwise, you can just let a parser generator do it for you.

To see how the lookahead computations fit into the grammar to parser mapping, let's build a parser for the nested list-of-names language in Section 2.4, *Tokenizing Sentences*, on page 43.

Implementation

We're going to build a parser, `ListParser`, to go with the lexer from Pattern 2, *LL(1) Recursive-Descent Lexer*, on page 49. Here's the grammar again:

[Download](#) `parsing/recursive-descent/NameList.g`

```
list      : '[' elements ']' ; // match bracketed list
elements : element (',' element)* ; // match comma-separated list
element  : NAME | list ; // element is name or nested list
```

Following the grammar-to-parser mapping we've established, we arrive at the following parser:

```
Download parsing/recursive-descent/ListParser.java

public class ListParser extends Parser {
    public ListParser(Lexer input) { super(input); }

    /** list : '[' elements ']' ; // match bracketed list */
    public void list() {
        match(ListLexer.LBRACK); elements(); match(ListLexer.RBRACK);
    }
    /** elements : element (',' element)* ; */
    void elements() {
        element();
        while ( lookahead.type==ListLexer.COMMA ) {
            match(ListLexer.COMMA); element();
        }
    }
    /** element : name | list ; // element is name or nested list */
    void element() {
        if ( lookahead.type==ListLexer.NAME ) match(ListLexer.NAME);
        else if ( lookahead.type==ListLexer.LBRACK ) list();
        else throw new Error("expecting name or list; found "+lookahead);
    }
}
```

Rules **elements** and **element** use lookahead to make parsing decisions. In **elements**, **COMMA** predicts entering the (...) *** subrule. In **element**, **NAME** predicts the first alternative, and **LBRACK** predicts the second alternative.

To support this concrete class, we need to build some support code in an abstract Parser class. First, we need two state variables: an input token stream and a lookahead buffer. In this case, we can use a single lookahead variable, token:

```
Download parsing/recursive-descent/Parser.java

Lexer input;      // from where do we get tokens?
Token lookahead;  // the current lookahead token
```

Alternatively, we could use a big token buffer that holds all tokens. In this way, we could track an index into the token buffer rather than a token field. For this particular implementation of this pattern, we'll assume that we cannot buffer all the input (we might be reading from a socket).

Next, we need methods to compare expected tokens against the lookahead symbol and to consume input.

[Download](#) parsing/recursive-descent/Parser.java

```

/** If lookahead token type matches x, consume & return else error */
public void match(int x) {
    if ( lookahead.type == x ) consume();
    else throw new Error("expecting "+input.getTokenName(x)+
        "; found "+ lookahead);
}
public void consume() { lookahead = input.nextToken(); }

```

To test our parser, we need a test rig like the following:

[Download](#) parsing/recursive-descent/Test.java

```

ListLexer lexer = new ListLexer(args[0]); // parse command-line arg
ListParser parser = new ListParser(lexer);
parser.list(); // begin parsing at rule list

```

Upon valid input, the test rig emits nothing because we don't have any application-specific code in there. We could easily add actions to track a list of names, for example. Upon error, we want to throw an exception like the following:

```

$ java Test '[a, ]'
Exception in thread "main" java.lang.Error:
    expecting name or list; found <']',RBRACK>
    at ListParser.element(ListParser.java:24)
    at ListParser.elements(ListParser.java:16)
    at ListParser.list(ListParser.java:8)
    at Test.main(Test.java:6)
$

```

Building an *LL(1)* parser is the easiest way to learn about parsers. In practice, though, we really need more than a single token of lookahead. The next pattern describes how to build an *LL(k)* parser for $k > 1$.

Related Patterns

See Pattern 2, *LL(1) Recursive-Descent Lexer*, on page 49, Pattern 4, *LL(k) Recursive-Descent Parser*, and Pattern 5, *Backtracking Parser*, on page 71.

4

LL(k) Recursive-Descent Parser



Purpose

This pattern analyzes the syntactic structure of the token sequence of a phrase using $k > 1$ lookahead tokens.

An $LL(k)$ parser augments the previous pattern with up to a fixed number of lookahead tokens, k .

Discussion

The strength of a recursive-descent parser depends entirely on the strength of its lookahead decisions. A single token of lookahead is pretty weak in that we usually have to contort grammars to make them $LL(1)$. By allowing a larger (but still fixed) lookahead buffer, we get a parser strong enough for most computer languages. This includes configuration files, data formats, network protocols, graphics languages, and many programming languages. Some programming languages present tougher challenges, though, for which we'll need a more powerful recognition pattern. Pattern 5, *Backtracking Parser*, on page 71 introduces an extension to this fixed-lookahead recursive-descent recognizer that, in effect, allows arbitrary lookahead.

Having more lookahead is like being able to see farther down multiple paths emanating from a fork in a maze. The farther we can see ahead, the easier it is to decide which path to take. More powerful parsing decisions make it easier to build parsers. We don't have to contort our parsers (or grammars) as much to suit a weak underlying parsing strategy.

In this Discussion section, we're going to figure out why we need more than a single token of lookahead and figure out how to build circular lookahead buffers. In the Implementation section, we'll use that lookahead buffer in an $LL(k)$ parser.

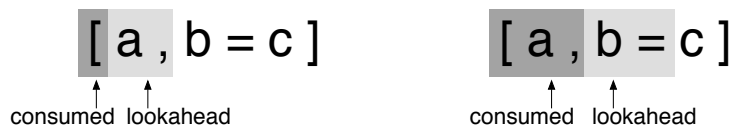
Motivating the Need for More Lookahead

To see why we need $k > 1$ lookahead, let's augment the list-of-names grammar from Section 2.4, *Tokenizing Sentences*, on page 43 to allow assignments as list elements. For example, we want to recognize input such as `[a, b=c, [d,e]]`. To accommodate this change, we can add an alternative to **element** that matches assignments:

```
list      : '[' elements ']' ;           // match bracketed list
elements : element (',' element)* ;    // match comma-separated list
element  : NAME '=' NAME                // match assignment such as a=b
          | NAME
          | list
          ;
```

The new alternative renders **element** non- $LL(1)$ since the first two alternatives start with the same **NAME** token. Now, we need two lookahead

tokens to distinguish the alternatives. The grammar is *LL*(2). Each time we look for an **element**, we need to decide whether it's an assignment or just a name. If the lookahead sequence is **NAME** followed by **=**, the parsing decision should predict the first alternative (an assignment). Otherwise, the parser should predict the second alternative. For example, the following diagram represents the lookahead available to the parsing decision in **element** for input [a,b=c]:



If we had only an *LL*(1) parser, we'd have to rewrite **element** to look like this:

```

element : NAME ('=' NAME)? // match assignment such as a=b or just a
        | list
        ;

```

This (..)? optional subrule version works but is less clear. There are lots of similar situations that occur in real grammars.

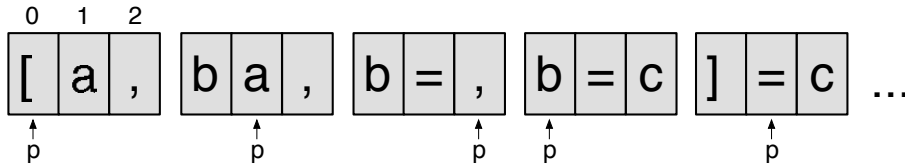
Building a Circular Lookahead Buffer

The simplest way to provide lots of parser lookahead is to buffer up all the input tokens. The input cursor can be an integer index, say, *p* into that buffer. We can execute *p*++ to consume tokens. The next *k* tokens of lookahead would be *tokens*[*p*..*tokens*[*p*+*k*-1]]. This approach works for finite and reasonably small input. This obviously doesn't work for infinite token streams like network sockets.

When we can't buffer up all of the input, we need to make a *k*-sized buffer of tokens. We'll have at most *k* tokens in memory at once. Our index, *p*, now moves only through the fixed lookahead buffer. To consume a token, we increment *p* and add a token to the end of the buffer.

The only complication is that the buffer size is fixed so we have to add tokens in a circular fashion: *p* ranges from 0..*k*-1. A circular buffer is one where indexes that fall off the end wrap to the beginning. In other words, in a circular buffer of, say, three tokens, index 2 is the last valid index. Index *p*=3 actually wraps to become index 0. Modulo expression *p*%3 wraps indexes nicely for a buffer of size 3.

Graphically, here is how p moves through the lookahead buffer as the parser consumes tokens from input sentence $[a,b=c]$:



When we start parsing, the lookahead buffer looks like the leftmost diagram where $p=0$. The second diagram (where $p=1$) shows the buffer after we've consumed the first $[$ token. We add tokens behind p as we advance. The rightmost diagram shows the state of the lookahead buffer after we've loaded the final $]$ token. p points at $=$, and so the parser needs to consume three more times before running out of input (those p positions aren't shown).

Implementation

Let's implement the list-of-names language augmented with assignments from the previous section. First we'll build the lookahead infrastructure, and then we'll implement *LL(2)* rule **element**.

To support a fixed lookahead buffer, our Parser support class needs the following fields:

```
Download parsing/multi/Parser.java

Lexer input;           // from where do we get tokens?
Token[] lookahead;     // circular lookahead buffer
int k;                 // how many lookahead symbols
int p = 0;             // circular index of next token position to fill
```

Because different parsers need different amounts of lookahead, the constructor takes a size argument and initializes the lookahead buffer:

```
Download parsing/multi/Parser.java

public Parser(Lexer input, int k) {
    this.input = input;
    this.k = k;
    lookahead = new Token[k];           // make lookahead buffer
    for (int i=1; i<=k; i++) consume(); // prime buffer with k lookahead
}
```

To consume a token, the parser advances the token index in a circular fashion and adds another token to the buffer.

[Download](#) parsing/multi/Parser.java

```
public void consume() {
    lookahead[p] = input.nextTok(); // fill next position with token
    p = (p+1) % k;                 // increment circular index
}
```

To isolate the lookahead mechanism from the parser, it's a good idea to create lookahead methods, LA() and LT(). Method LA() returns lookahead token types up to *k* symbols ahead starting at *k*=1. Recall that token types are integers representing input symbol categories. Method LT() returns the actual lookahead token at a particular lookahead depth.

[Download](#) parsing/multi/Parser.java

```
public Token LT(int i) {return lookahead[(p+i-1) % k];} // circular fetch
public int LA(int i) { return LT(i).type; }
public void match(int x) {
    if ( LA(1) == x ) consume();
    else throw new Error("expecting "+input.getTokenName(x)+
        "; found "+LT(1));
}
```

Only rule **element** needs more than single token of lookahead in our case. Its implementation method tests the first two tokens of lookahead, LA(1) and LA(2), to predict the first alternative:

[Download](#) parsing/multi/LookaheadParser.java

```
/** element : NAME '=' NAME | NAME | list ; assignment, NAME or list */
void element() {
    if ( LA(1)==LookaheadLexer.NAME && LA(2)==LookaheadLexer.EQUALS ) {
        match(LookaheadLexer.NAME);
        match(LookaheadLexer.EQUALS);
        match(LookaheadLexer.NAME);
    }
    else if ( LA(1)==LookaheadLexer.NAME ) match(LookaheadLexer.NAME);
    else if ( LA(1)==LookaheadLexer.LBRACK ) list();
    else throw new Error("expecting name or list; found "+LT(1));
}
```

Because the method tests the alternatives in order, the second alternative prediction expression gets away with looking only a single symbol ahead. Also note that a single lookahead, [, uniquely predicts the third alternative. The lookahead depth *k* in LL(*k*) is really a maximum not the exact, fixed amount of lookahead each parsing decision uses.

Our test rig is the same as that in Pattern 3, *LL(1) Recursive-Descent Parser*, on page 54 except that we need to pass in the necessary lookahead depth to the parser constructor.

[Download](#) parsing/multi/Test.java

```
LookaheadLexer lexer = new LookaheadLexer(args[0]); // parse arg
LookaheadParser parser = new LookaheadParser(lexer, 2);
parser.list(); // begin parsing at rule list
```

If we pass in a valid list such as [a,b=c,[d,e]], the test rig quietly returns this:

```
$ java Test '[a,b=c,[d,e]]'
$
```

Upon error, however, the parser throws an exception:

```
$ java Test '[a,b=c,,[d,e]]'
Exception in thread "main" java.lang.Error:
    expecting name or list; found <',',>
    at LookaheadParser.element(LookaheadParser.java:25)
    at LookaheadParser.elements(LookaheadParser.java:12)
    at LookaheadParser.list(LookaheadParser.java:7)
    at Test.main(Test.java:6)
$
```

Related Patterns

This pattern builds on Pattern 3, *LL(1) Recursive-Descent Parser*, on page 54 and uses Pattern 2, *LL(1) Recursive-Descent Lexer*, on page 49 to feed it tokens. Pattern 5, *Backtracking Parser*, on page 71 extends this pattern with arbitrary lookahead.

Up Next

At this point, we've got the most important parsing patterns down. To handle really tough language problems, we'll need to dress up the recursive-descent parser even further. The next chapter defines patterns for arbitrary lookahead and using semantic information to guide the parse.